

REPORT

HelixQA Bluff Audit — Definitive Report

Field	Value
Revision	1
Last modified	2026-06-10T17:00:00Z
Status	complete
Authority	Operator mandate 2026-06-10 (“is HelixQA the source of bluff?”)
Scope	helix_ota HelixQA incorporation + canonical HelixDevelopment/HelixQA engine (HEAD bca3b36)

0. VERDICT (FACT): HelixQA is NOT the source of bluff.

The canonical HelixQA engine scores a challenge PASS only on **(a)** a real dispatched-script exit 0 **AND (b)** a satisfied evidence ledger (real, non-empty artefacts), and it provably catches its own negation under a §1.1 paired mutation. The real gap in **helix_ota** is an **incorporation gap (§11.4.27)**, not a scoring bluff: HelixQA was never wired as a submodule, and the bank was a **declared-only manifest nothing machine-executed**. No challenge ever scored a *false* PASS — the dispatched scripts are genuinely anti-bluff and the committed evidence files are real captured runs.

1. Ground truth (verified)

- `.gitmodules` has **no HelixQA entry**; `tools/helixqa/` contained **only** `banks/helix_ota.yaml`. Repo-wide `grep` for the bank / `dispatch_command / challenges`: found **only documentation references** (`CLAUDE.md`, `coverage_ledger.md`, `spec_impl_alignment.md`) — **zero executables** consuming the bank (pre-fix). No `helixqa` on `PATH`.
- Canonical repo reachable: `gh repo view HelixDevelopment/HelixQA` OK; `clone HEAD bca3b3692217a89d59bf5c91c4d248553514071a`.

2. Per-challenge dispatch audit

All bank entries dispatch to **real**, **existing** anti-bluff scripts and cite **real**, **non-empty**, **committed** evidence:

Challenge(s)	Dispatches to	Evidence
HOTA-AUTH-LOGIN / DEVICE-REGISTER / GROUP-LIFECYCLE / AUDIT-TRAIL / TELEMETRY-OVERVIEW / ROLLOUT-ROUTE-GATES	tests/e2e/ challenge_operational.sh (404 ln)	tests/e2e/RUN_EVIDENCE.txt
HOTA-PIPELINE-SIGNED	tests/e2e/pipeline_signed.sh (401 ln)	PIPELINE_EVIDENCE.txt
HOTA-RECALL-LIFECYCLE	tests/e2e/recall_lifecycle.sh (424 ln)	RECALL_EVIDENCE.txt
HOTA-SECURITY-PROBES	tests/security/ security_probes.sh (371 ln)	tests/security/RUN_EVIDENCE.txt
HOTA-FILTERS-PAGINATION	tests/e2e/ challenge_filters_pagination.sh (513 ln)	FILTERS_PAGINATION_EVIDENCE.txt

Sampled challenge_operational.sh: real curl+jq, pass()/fail() counters, fatal()→exit 1, missing prereq→exit 3 SKIP, RESULT: FAIL/exit 1 when FAIL>0. Genuinely anti-bluff. No decorative / never-executed entry exists.

3. Prior-claims audit

No false-PASS claim found in docs/CONTINUATION.md / docs/changelogs/. The §11.4.25 coverage ledger marks banks/helix_ota.yaml:HOTA-* COVERED — accurate at the bank-declares-a-real-challenge level, but did **not disclose that nothing machine-executed the bank** (the one declared-vs-enforced drift the fix closes). RUN_EVIDENCE.txt is a real captured run (2026-06-08T15:27:01Z, committed 1b7d151 “real run 39/0/1 PASS”).

4. Engine audit (the crux) — pkg/testbank/dispatch.go @ bca3b36

Dispatcher.Run (dispatch.go:194): **GATE 1** dispatch exit — non-zero / spawn error → FAIL (:231–242); **GATE 2** evidence ledger (§11.4.69) — every RequiredEvidence token must resolve to a real **non-empty** artefact, any missing → FAIL even on exit 0 (:247–256); GlobEvidenceResolver rejects 0-byte files (:104–110); nil resolver = all-missing = FAIL (:270–272). The exact bluff (green script, absent evidence) is guarded by TestDispatcher_ScriptZeroExitButMissingEvidence_StillFails (dispatch_test.go:200–214). 14-case paired self-test present.

PROVEN with captured evidence (docs/research/helixqa_bluff_audit/): - dispatch_engine_green.txt — verbatim dispatch.go+conduit run in an isolated faithful module: **14/14 PASS**. - dispatch_engine_paired_mutation_FAIL.txt — §1.1 mutation forcing missing = nil → the ledger tests **FAIL** (expected FAIL, got PASS); restored → GREEN. The gate catches its own negation.

Honest residual (FACT, not a bluff): the case-level Dispatcher evidence-ledger is sound + self-tested but **not yet wired into the autonomous session runner** (referenced only by dispatch.go/schema.go/tests). The path that IS wired today is the per-step ActionTypeShell executor (schema.go:344, consumed by pkg/orchestrator/definition_challenge.go + pkg/autonomous/structured_executor.go), running action: "shell: <cmd>" via real os/exec — its comment records it **already closed a prior runner bluff HXC-011** (“desktop-platform bank cases loaded but never executed — a §11.4/CONST-035 PASS-bluff in the QA runner itself”). HelixQA found+fixed a bluff in its own runner; the Dispatcher is the next-stronger seam awaiting end-to-end wiring (§11.4.124 sound-but-unwired — recommended follow-up, NOT a current bluff).

5. Deep web research (§11.4.8 / §11.4.99) — the fix is grounded, not invented

- **Mutation testing** = “test your tests”: a surviving mutant ⇒ verification gap; killing it proves the suite catches the negation (Stryker/PIT). [JavaPro 2026-01-21](#), [MS Learn](#), [Calmops](#).
- **Assertion-free / non-exercising anti-pattern**: a test without clear assertions can pass with a defect present. [Google Testing Blog 2009-02](#), [arXiv 2005.05359](#), [SWE at Google ch12](#).
- **Characterization / golden-master + self-testing code**: observed I/O is the golden master; the test FAILs on drift. [Fowler SelfTestingCode](#).
- **AOSP CTS/Tradefed evidence model**: verdicts recorded in test_result.xml + test-record.pb; **no result file ⇒ untrustworthy run** — the same “evidence must exist & be non-empty” rule as the ledger. [AOSP interpret CTS](#).

6. What was fixed (helix_ota side) + captured proof

New `tools/helixqa/run_bank.sh` (+ companion docs/scripts/run_bank.md, §11.4.18): machine-executes the bank with the SAME two gates as the canonical engine. Modes: `--dry-run` (static audit, no live server — respects §11.4.119 port-contention deferral), `--self-test` (§1.1 paired mutation of its own ledger), LIVE full-bank.

- `run_bank_dry_run.txt` — **10/0/0 PASS** (every challenge → real dispatch + non-empty evidence).
- `run_bank_self_test.txt` — **SELF-TEST PASS**: the missing-evidence bank correctly FAILs (gate fired), the real-evidence bank PASSes; deterministic **3/3** (§11.4.50). Negative control (repoint one `evidence_artifact` at a missing file) → **9 passed / 1 failed, RESULT: FAIL**.
- Two real runner bugs were caught *by the self-test itself* during development (absolute-path resolution; `set -u` trap scope) and fixed at source (§11.4.1).
- `.gitignore` updated with `.helixqa_selftest.*` / scratch ignore.

Conductor independent re-confirmation (2026-06-10): `--dry-run` → 10/0/0 PASS; `--self-test` → SELF-TEST PASS, re-run by the conductor (structurally separate from the authoring agent, §11.4.142).

7. Recommended follow-ups (tracked — NOT done in this audit)

1. Wire HelixQA as a submodule per §11.4.27 (pointer = bca3b36, `install_upstreams` §11.4.36, CodeGraph own-org inclusion §11.4.79).
2. Wire the case-level Dispatcher evidence-ledger into the autonomous runner (§11.4.124 sound-but-unwired).
3. Add `run_bank.sh --self-test` to the pre-build gate.
4. Translate the bank to HelixQA's native `test_cases: schema (dispatches_to: + required_evidence:)`.
5. Run the LIVE full-bank once streams are quiescent (deferred for §11.4.119 port contention during the parallel wave).

8. Push status

Nothing was pushed by the investigation. No commits were created in the HelixQA scratch clone — the engine was audited read-only; **no HelixQA upstream push is required by this investigation** (the canonical engine is already correct at bca3b36). The `helix_ota`-side runner + bank + evidence are committed by the conductor to the parent repo's 4 upstreams (fast-forward only, no force-push §11.4.113).